JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class SimpleDateFormat

java.lang.Object
    java.text.Format
        java.text.DateFormat
            java.text.SimpleDateFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

`public class` **`SimpleDateFormat`**
`extends` `DateFormat`

`SimpleDateFormat` is a concrete class for formatting and parsing dates in a locale-sensitive manner. It allows for formatting (date → text), parsing (text → date), and normalization.

`SimpleDateFormat` allows you to start by choosing any user-defined patterns for date-time formatting. However, you are encouraged to create a date-time formatter with either `getTimeInstance`, `getDateInstance`, or `getDateTimeInstance` in `DateFormat`. Each of these class methods can return a date/time formatter initialized with a default format pattern. You may modify the format pattern using the `applyPattern` methods as desired. For more information on using these methods, see `DateFormat`.

Date and Time Patterns

Date and time formats are specified by *date and time pattern* strings. Within date and time pattern strings, unquoted letters from `'A'` to `'Z'` and from `'a'` to `'z'` are interpreted as pattern letters representing the components of a date or time string. Text can be quoted using single quotes (`'`) to avoid interpretation. `"''"` represents a single quote. All other characters are not interpreted; they're simply copied into the output string during formatting or matched against the input string during parsing.

The following pattern letters are defined (all other characters from `'A'` to `'Z'` and from `'a'` to `'z'` not in the table below are reserved). `applyPattern(String)`, `applyLocalizedPattern(String)`, and the `SimpleDateFormat constructors` throw `IllegalArgumentException` when passed a pattern containing an unquoted reserved character.

Letter	Date or Time Component	Presentation	Examples
`G`	Era designator	Text	`AD`
`y`	Year	Year	`1996; 96`
`Y`	Week year	Year	`2009; 09`

Letter	Date or Time Component	Presentation	Examples
`M`	Month in year (context sensitive)	Month	`July`; `Jul`; `07`
`L`	Month in year (standalone form)	Month	`July`; `Jul`; `07`
`w`	Week in year	Number	`27`
`W`	Week in month	Number	`2`
`D`	Day in year	Number	`189`
`d`	Day in month	Number	`10`
`F`	Day of week in month	Number	`2`
`E`	Day name in week	Text	`Tuesday`; `Tue`
`u`	Day number of week (1 = Monday, ..., 7 = Sunday)	Number	`1`
`a`	Am/pm marker	Text	`PM`
`H`	Hour in day (0-23)	Number	`0`
`k`	Hour in day (1-24)	Number	`24`
`K`	Hour in am/pm (0-11)	Number	`0`
`h`	Hour in am/pm (1-12)	Number	`12`
`m`	Minute in hour	Number	`30`
`s`	Second in minute	Number	`55`
`S`	Millisecond	Number	`978`
`z`	Time zone	General time zone	`Pacific Standard Time`; `PST`; `GMT-08:00`
`Z`	Time zone	RFC 822 time zone	`-0800`
`X`	Time zone	ISO 8601 time zone	`-08`; `-0800`; `-08:00`

Pattern letters are usually repeated, as their number determines the exact presentation:

- **Text:** For formatting, if the number of pattern letters is 4 or more, the full form is used; otherwise a short or abbreviated form is used if available. For parsing, both forms are accepted, independent of the number of

pattern letters.

- **Number:** For formatting, the number of pattern letters is the minimum number of digits, and shorter numbers are zero-padded to this amount. For parsing, the number of pattern letters is ignored unless it's needed to separate two adjacent fields.

- **Year:** If the formatter's `Calendar` is the Gregorian calendar, the following rules are applied.
    - For formatting, if the number of pattern letters is 2, the year is truncated to 2 digits; otherwise it is interpreted as a number.
    - For parsing, if the number of pattern letters is more than 2, the year is interpreted literally, regardless of the number of digits. So using the pattern "MM/dd/yyyy", "01/11/12" parses to Jan 11, 12 A.D.
    - For parsing with the abbreviated year pattern ("y" or "yy"), `SimpleDateFormat` must interpret the abbreviated year relative to some century. It does this by adjusting dates to be within 80 years before and 20 years after the time the `SimpleDateFormat` instance is created. For example, using a pattern of "MM/dd/yy" and a `SimpleDateFormat` instance created on Jan 1, 1997, the string "01/11/12" would be interpreted as Jan 11, 2012 while the string "05/04/64" would be interpreted as May 4, 1964. During parsing, only strings consisting of exactly two digits, as defined by `Character.isDigit(char)`, will be parsed into the default century. Any other numeric string, such as a one digit string, a three or more digit string, or a two digit string that isn't all digits (for example, "-1"), is interpreted literally. So "01/02/3" or "01/02/003" are parsed, using the same pattern, as Jan 2, 3 AD. Likewise, "01/02/-3" is parsed as Jan 2, 4 BC.

  Otherwise, calendar system specific forms are applied. For both formatting and parsing, if the number of pattern letters is 4 or more, a calendar specific long form is used. Otherwise, a calendar specific short or abbreviated form is used.

  If week year `'Y'` is specified and the calendar doesn't support any week years, the calendar year (`'y'`) is used instead. The support of week years can be tested with a call to `getCalendar().isWeekDateSupported()`.

- **Month:** If the number of pattern letters is 3 or more, the month is interpreted as text; otherwise, it is interpreted as a number.
    - Letter *M* produces context-sensitive month names, such as the embedded form of names. Letter *M* is context-sensitive in the sense that when it is used in the standalone pattern, for example, "MMMM", it gives the standalone form of a month name and when it is used in the pattern containing other field(s), for example, "d

MMMM", it gives the format form of a month name. For example, January in the Catalan language is "de gener" in the format form while it is "gener" in the standalone form. In this case, "MMMM" will produce "gener" and the month part of the "d MMMM" will produce "de gener". If a `DateFormatSymbols` has been set explicitly with constructor `SimpleDateFormat(String,DateFormatSymbols)` or method `setDateFormatSymbols(DateFormatSymbols)`, the month names given by the `DateFormatSymbols` are used.
    - Letter *L* produces the standalone form of month names.

- **General time zone:** Time zones are interpreted as text if they have names. For time zones representing a GMT offset value, the following syntax is used:

```
GMTOffsetTimeZone:
        GMT Sign Hours : Minutes
Sign: one of
        + -
Hours:
        Digit
        Digit Digit
Minutes:
        Digit Digit
Digit: one of
        0 1 2 3 4 5 6 7 8 9
```

  Hours must be between 0 and 23, and *Minutes* must be between 00 and 59. The format is locale independent and digits must be taken from the Basic Latin block of the Unicode standard.

  For parsing, RFC 822 time zones are also accepted.

- **RFC 822 time zone:** For formatting, the RFC 822 4-digit time zone format is used:

```
RFC822TimeZone:
        Sign TwoDigitHours Minutes
TwoDigitHours:
        Digit Digit
```

  TwoDigitHours must be between 00 and 23. Other definitions are as for general time zones.

  For parsing, general time zones are also accepted.

- **ISO 8601 Time zone:** The number of pattern letters designates the format for both formatting and parsing as follows:

```
ISO8601TimeZone:
        OneLetterISO8601TimeZone
        TwoLetterISO8601TimeZone
        ThreeLetterISO8601TimeZone
OneLetterISO8601TimeZone:
        Sign TwoDigitHours
        Z
TwoLetterISO8601TimeZone:
        Sign TwoDigitHours Minutes
        Z
ThreeLetterISO8601TimeZone:
        Sign TwoDigitHours : Minutes
        Z
```

  Other definitions are as for general time zones or RFC 822 time zones.

  For formatting, if the offset value from GMT is 0, `"Z"` is produced. If the number of pattern letters is 1, any fraction of an hour is ignored. For example, if the pattern is `"X"` and the time zone is `"GMT+05:30"`, `"+05"` is produced.

  For parsing, `"Z"` is parsed as the UTC time zone designator. General time zones are *not* accepted.

  If the number of pattern letters is 4 or more, `IllegalArgumentException` is thrown when constructing a `SimpleDateFormat` or applying a pattern.

`SimpleDateFormat` also supports *localized date and time pattern* strings. In these strings, the pattern letters described above may be replaced with other, locale dependent, pattern letters. `SimpleDateFormat` does not deal with the localization of text other than the pattern letters; that's up to the client of the class.

Examples

The following examples show how date and time patterns are interpreted in the U.S. locale. The given date and time are 2001-07-04 12:08:56 local time in the U.S. Pacific Time time zone.

Date and Time Pattern	Result
`"yyyy.MM.dd G 'at' HH:mm:ss z"`	`2001.07.04 AD at 12:08:56 PDT`

Date and Time Pattern	Result
"EEE, MMM d, ''yy"	Wed, Jul 4, '01
"h:mm a"	12:08 PM
"hh 'o''clock' a, zzzz"	12 o'clock PM, Pacific Daylight Time
"K:mm a, z"	0:08 PM, PDT
"yyyyy.MMMMM.dd GGG hh:mm aaa"	02001.July.04 AD 12:08 PM
"EEE, d MMM yyyy HH:mm:ss Z"	Wed, 4 Jul 2001 12:08:56 -0700
"yyMMddHHmmssZ"	010704120856-0700
"yyyy-MM-dd'T'HH:mm:ss.SSSZ"	2001-07-04T12:08:56.235-0700
"yyyy-MM-dd'T'HH:mm:ss.SSSXXX"	2001-07-04T12:08:56.235-07:00
"YYYY-'W'ww-u"	2001-W27-3

Synchronization

Date formats are not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

API Note:

Consider using `DateTimeFormatter` as an immutable and thread-safe alternative.

Since:

1.1

See Also:

Java Tutorial, `Calendar`, `TimeZone`, `DateFormat`, `DateFormatSymbols`, `DateTimeFormatter`, Serialized Form

Nested Class Summary

Nested classes/interfaces declared in class DateFormat
`DateFormat.Field`

Field Summary

Fields declared in class DateFormat

AM_PM_FIELD, calendar, DATE_FIELD, DAY_OF_WEEK_FIELD, DAY_OF_WEEK_IN_MONTH_FIELD, DAY_OF_YEAR_FIELD, DEFAULT, ERA_FIELD, FULL, HOUR_OF_DAY0_FIELD, HOUR_OF_DAY1_FIELD, HOUR0_FIELD, HOUR1_FIELD, LONG, MEDIUM, MILLISECOND_FIELD, MINUTE_FIELD, MONTH_FIELD, numberFormat, SECOND_FIELD, SHORT, TIMEZONE_FIELD, WEEK_OF_MONTH_FIELD, WEEK_OF_YEAR_FIELD, YEAR_FIELD

Constructor Summary

Constructors

Constructor	Description
SimpleDateFormat()	Constructs a SimpleDateFormat using the default pattern and date format symbols for the default FORMAT locale.
SimpleDateFormat(String pattern)	Constructs a SimpleDateFormat using the given pattern and the default date format symbols for the default FORMAT locale.
SimpleDateFormat(String pattern, DateFormatSymbols formatSymbols)	Constructs a SimpleDateFormat using the given pattern and date format symbols.
SimpleDateFormat(String pattern, Locale locale)	Constructs a SimpleDateFormat using the given pattern and the default date format symbols for the given locale.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	applyLocalizedPattern (String pattern)	Applies the given localized pattern string to this date format.

void	**applyPattern**(String pattern)	Applies the given pattern string to this date format.
Object	**clone**()	Creates a copy of this SimpleDateFormat.
boolean	**equals**(Object obj)	Compares the specified object with this SimpleDateFormat for equality.
StringBuffer	**format**(Date date, StringBuffer toAppendTo, FieldPosition pos)	Formats the given Date into a date/time string and appends the result to the given StringBuffer.
AttributedCharacterI	**formatToCharacterIterato**(Object obj)	Formats an Object producing an AttributedCharacterIterator.
Date	**get2DigitYearStart**()	Returns the beginning date of the 100-year period 2-digit years are interpreted as being within.
DateFormatSymbols	**getDateFormatSymbols**()	Gets a copy of the date and time format symbols of this date format.
int	**hashCode**()	Returns the hash code value for this SimpleDateFormat.
Date	**parse**(String text, ParsePosition pos)	Parses text from a string to produce a Date.
void	**set2DigitYearStart**(Date startDate)	Sets the 100-year period 2-digit years will be interpreted as being in to begin on the date the user specifies.
void	**setDateFormatSymbols**(DateFormatSymbols newFo	

		Sets the date and time format symbols of this date format.
String	**toLocalizedPattern**()	Returns a localized pattern string describing this date format.
String	**toPattern**()	Returns a pattern string describing this date format.
String	**toString**()	Returns a string identifying this SimpleDateFormat, for debugging.

Methods declared in class DateFormat

format, format, getAvailableLocales, getCalendar, getDateInstance, getDateInstance, getDateInstance, getDateTimeInstance, getDateTimeInstance, getDateTimeInstance, getInstance, getNumberFormat, getTimeInstance, getTimeInstance, getTimeInstance, getTimeZone, isLenient, parse, parseObject, setCalendar, setLenient, setNumberFormat, setTimeZone

Methods declared in class Format

format, parseObject

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

SimpleDateFormat

```
public SimpleDateFormat()
```

Constructs a SimpleDateFormat using the default pattern and date format symbols for the default FORMAT locale. **Note:** This constructor may not support all locales. For full coverage, use the factory methods in the DateFormat class.

SimpleDateFormat

```
public SimpleDateFormat(String pattern)
```

Constructs a SimpleDateFormat using the given pattern and the default date format symbols for the default FORMAT locale. **Note:** This constructor may not support all locales. For full coverage, use the factory methods in the DateFormat class.

This is equivalent to calling SimpleDateFormat(pattern, Locale.getDefault(Locale.Category.FORMAT)).

Parameters:

pattern - the pattern describing the date and time format

Throws:

NullPointerException - if the given pattern is null

IllegalArgumentException - if the given pattern is invalid

See Also:

Locale.getDefault(java.util.Locale.Category), Locale.Category.FORMAT

SimpleDateFormat

```
public SimpleDateFormat(String pattern,
                        Locale locale)
```

Constructs a SimpleDateFormat using the given pattern and the default date format symbols for the given locale. **Note:** This constructor may not support all locales. For full coverage, use the factory methods in the DateFormat class.

Parameters:

pattern - the pattern describing the date and time format

locale - the locale whose date format symbols should be used

Throws:

NullPointerException - if the given pattern or locale is null

IllegalArgumentException - if the given pattern is invalid

SimpleDateFormat

```
public SimpleDateFormat(String pattern,
                        DateFormatSymbols formatSymbols)
```

Constructs a SimpleDateFormat using the given pattern and date format symbols.

Parameters:

`pattern` - the pattern describing the date and time format

`formatSymbols` - the date format symbols to be used for formatting

Throws:

`NullPointerException` - if the given pattern or formatSymbols is null

`IllegalArgumentException` - if the given pattern is invalid

Method Details

set2DigitYearStart

```
public void set2DigitYearStart(Date startDate)
```

Sets the 100-year period 2-digit years will be interpreted as being in to begin on the date the user specifies.

Parameters:

`startDate` - During parsing, two digit years will be placed in the range `startDate` to `startDate + 100 years`.

Throws:

`NullPointerException` - if `startDate` is `null`.

Since:

1.2

See Also:

`get2DigitYearStart()`

get2DigitYearStart

```
public Date get2DigitYearStart()
```

Returns the beginning date of the 100-year period 2-digit years are interpreted as being within.

Returns:

the start of the 100-year period into which two digit years are parsed

Since:

1.2

See Also:

`set2DigitYearStart(Date)`

format

```
public StringBuffer format(Date date,
                           StringBuffer toAppendTo,
                           FieldPosition pos)
```

Formats the given `Date` into a date/time string and appends the result to the given `StringBuffer`.

Specified by:

`format` in class `DateFormat`

Parameters:

`date` - the date-time value to be formatted into a date-time string.

`toAppendTo` - where the new date-time text is to be appended.

`pos` - keeps track on the position of the field within the returned string. For example, given a date-time text `"1996.07.10 AD at 15:08:56 PDT"`, if the given `fieldPosition` is `DateFormat.YEAR_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 4, respectively. Notice that if the same date-time field appears more than once in a pattern, the `fieldPosition` will be set for the first occurrence of that date-time field. For instance, formatting a `Date` to the date-time string `"1 PM PDT (Pacific Daylight Time)"` using the pattern `"h a z (zzzz)"` and the alignment field `DateFormat.TIMEZONE_FIELD`, the begin index and end index of `fieldPosition` will be set to 5 and 8, respectively, for the first occurrence of the timezone pattern character `'z'`.

Returns:

the formatted date-time string.

Throws:

`NullPointerException` - if any of the parameters is `null`.

formatToCharacterIterator

```
public AttributedCharacterIterator formatToCharacterIterator
(Object obj)
```

Formats an Object producing an `AttributedCharacterIterator`. You can use the returned `AttributedCharacterIterator` to build the resulting String, as well as to determine information about the resulting String.

Each attribute key of the AttributedCharacterIterator will be of type `DateFormat.Field`, with the corresponding attribute value being the same as the attribute key.

Overrides:

formatToCharacterIterator in class Format

Parameters:

obj - The object to format

Returns:

AttributedCharacterIterator describing the formatted value.

Throws:

NullPointerException - if obj is null.

IllegalArgumentException - if the Format cannot format the given object, or if the Format's pattern string is invalid.

Since:

1.4

parse

```
public Date parse(String text,
                  ParsePosition pos)
```

Parses text from a string to produce a Date.

The method attempts to parse text starting at the index given by pos. If parsing succeeds, then the index of pos is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed date is returned. The updated pos can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of pos is not changed, the error index of pos is set to the index of the character where the error occurred, and null is returned.

This parsing operation uses the calendar to produce a Date. All of the calendar's date-time fields are cleared before parsing, and the calendar's default values of the date-time fields are used for any missing date-time information. For example, the year value of the parsed Date is 1970 with GregorianCalendar if no year value is given from the parsing operation. The TimeZone value may be overwritten, depending on the given pattern and the time zone value in text. Any TimeZone value that has previously been set by a call to setTimeZone may need to be restored for further operations.

Specified by:

parse in class DateFormat

Parameters:

text - A String, part of which should be parsed.

pos - A ParsePosition object with index and error index information as described above.

Returns:

A `Date` parsed from the string. In case of error, returns null.

Throws:

`NullPointerException` - if `text` or `pos` is null.

toPattern

```
public String toPattern()
```

Returns a pattern string describing this date format.

Returns:

a pattern string describing this date format.

toLocalizedPattern

```
public String toLocalizedPattern()
```

Returns a localized pattern string describing this date format.

Returns:

a localized pattern string describing this date format.

applyPattern

```
public void applyPattern(String pattern)
```

Applies the given pattern string to this date format.

Parameters:

`pattern` - the new date and time pattern for this date format

Throws:

`NullPointerException` - if the given pattern is null

`IllegalArgumentException` - if the given pattern is invalid

applyLocalizedPattern

```
public void applyLocalizedPattern(String pattern)
```

Applies the given localized pattern string to this date format.

Parameters:

`pattern` - a String to be mapped to the new date and time format pattern for this format

Throws:

`NullPointerException` - if the given pattern is null

IllegalArgumentException - if the given pattern is invalid

getDateFormatSymbols

```
public DateFormatSymbols getDateFormatSymbols()
```

Gets a copy of the date and time format symbols of this date format.

Returns:

the date and time format symbols of this date format

See Also:

setDateFormatSymbols(DateFormatSymbols)

setDateFormatSymbols

```
public void setDateFormatSymbols(DateFormatSymbols newFormatSymbols)
```

Sets the date and time format symbols of this date format.

Parameters:

newFormatSymbols - the new date and time format symbols

Throws:

NullPointerException - if the given newFormatSymbols is null

See Also:

getDateFormatSymbols()

clone

```
public Object clone()
```

Creates a copy of this SimpleDateFormat. This also clones the format's date format symbols.

Overrides:

clone in class DateFormat

Returns:

a clone of this SimpleDateFormat

See Also:

Cloneable

hashCode

```
public int hashCode()
```

Returns the hash code value for this SimpleDateFormat.

Overrides:

hashCode in class DateFormat

Implementation Requirements:

This method calculates the hash code value using the value returned by toPattern().

Returns:

the hash code value for this SimpleDateFormat

See Also:

Object.hashCode()

toString

```
public String toString()
```

Returns a string identifying this SimpleDateFormat, for debugging.

Overrides:

toString in class Object

Returns:

a string identifying this SimpleDateFormat, for debugging

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this SimpleDateFormat for equality. Returns true if the object is also a SimpleDateFormat and the two formats would format any value the same.

Overrides:

equals in class DateFormat

Implementation Requirements:

This method performs an equality check with a notion of class identity based on getClass(), rather than instanceof. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

obj - object to be compared for equality

Returns:

true if the specified object is equal to this SimpleDateFormat

See Also:

Object.equals(Object)

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.